Clean Architecture and CQRS

Contents

Clean Architecture and CQRS **1**

Scope . . . 1
Official requirements . . . 1
Project policy . . . 1
Checklist . . . 3
References . . . 3
Related . . . 3

Clean Architecture and CQRS

Scope

Project layout, dependency direction and request handling for the backend. Decision and rationale: `brain/decisions/ADR-0001-backend-dotnet-narchitecture.md`.

Official requirements

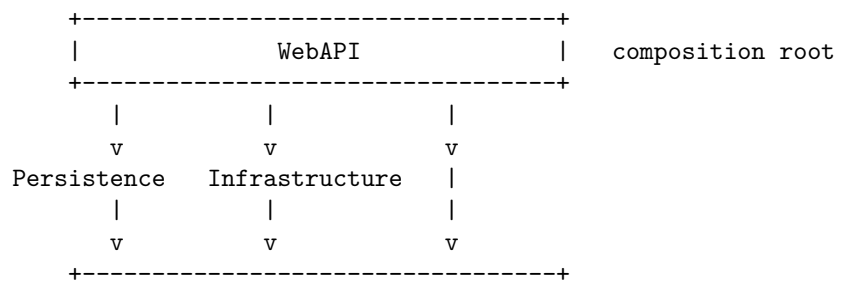
The subject mandates a client/server architecture with the backend as the single source of truth, and requires the architecture to be explained and defended at evaluation. Clean Architecture itself is a project choice, not an external mandate.

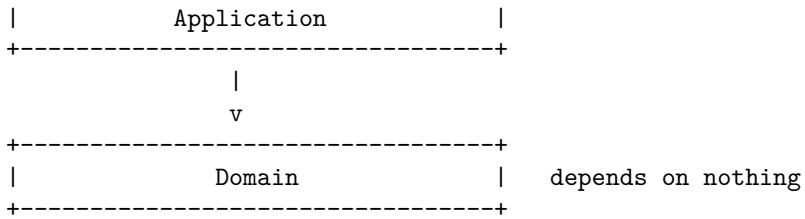
Project policy

Where the code lives

- Every backend project, file and test lives under **`MusicRoom/`**. This is the only write target.
- `reference/nArchitecture/` is read-only study material. Do not edit it, do not build it, do not add a `ProjectReference` to it, and do not copy files out of it into `MusicRoom/`. Read a pattern there, then write your own implementation. Its package versions are net6-era and subject IV.1 forbids committing third-party source.
- `reference/` is git-ignored and excluded from search and graph.

Projects and dependency direction





Project	Contains	May reference
`Domain`	entities, value objects, enums, domain exceptions, domain rules	nothing
`Application`	commands, queries, handlers, validators, pipeline behaviours, repository and service **interfaces**, DTOs	Domain
`Persistence`	`DbContext`, entity configurations, repository implementations, migrations	Application, Domain
`Infrastructure`	YouTube client, mail, token services, other external adapters	Application, Domain
`WebAPI`	controllers, SignalR hubs, middleware, DI registration, OpenAPI	all

Hard rules:

- `Domain` MUST NOT reference any other project or any infrastructure package.
- `Application` MUST NOT reference EF Core, ASP.NET Core, or any provider SDK.
- `WebAPI` MUST NOT touch `DbContext` directly.
- Dependencies point inward. A reference in the other direction is a build-breaking defect.

CQRS

- Every use case is one command or one query with one handler. No shared "service" god classes.
- Commands mutate and return the minimum needed (an id, or nothing). Queries never mutate.
- Handlers are thin: orchestrate, delegate rules to the domain, persist, return.
- A handler is the only place a transaction boundary is opened.
- REST controllers and SignalR hubs both dispatch to the **same** handler. Business logic never lives in a transport class.

Pipeline behaviours

Implemented with MediatR ([[ADR-0021-backend-dependency-set]]); validation uses FluentValidation. Registered once, in this order:

1. Request logging
2. Authorisation (ownership + licence/visibility check)
3. Validation (FluentValidation)
4. Transaction (commands only)
5. Caching (queries only, where justified)

A cross-cutting concern that appears in more than two handlers belongs in a behaviour.

What we take from nArchitecture

The layer split, the behaviour pipeline idea, the generic repository shape. Not its package versions, its Elasticsearch package, or its SQL Server coupling. See `brain/projects/reference-narchitecture.md`.

Checklist

- ☐ No outward project reference anywhere
- ☐ One handler per use case, named `<Verb><Noun>Command/Query`
- ☐ Controllers and hubs contain no branching business logic
- ☐ Cross-cutting concern implemented once, as a behaviour

References

- Clean Architecture, Robert C. Martin — the dependency rule
- .NET application architecture guides — https://learn.microsoft.com/dotnet/architecture/
- CQRS pattern — https://learn.microsoft.com/azure/architecture/patterns/cqrs

Related

`../dotnet-csharp/STANDARD.md` · `../ef-core-persistence/STANDARD.md` · `../signalr-realtime/STANDARD.md`